

1. One-Lane Bridge Synchronization

Problem

A bridge can accommodate a maximum of **3 vehicles simultaneously**. Vehicles from **East and West directions cannot be on the bridge at the same time**.

Create one thread for each vehicle.

Sample Input

Number of vehicles: 6

Vehicle	Direction
---------	-----------

V1	East
----	------

V2	East
----	------

V3	West
----	------

V4	East
----	------

V5	West
----	------

V6	West
----	------

Sample Output

V1 entered the bridge from East

V2 entered the bridge from East

V4 entered the bridge from East

V1 exited the bridge

V2 exited the bridge

V4 exited the bridge

V3 entered the bridge from West

V5 entered the bridge from West

V6 entered the bridge from West

All vehicles crossed successfully.

Students must use

- `pthread_create()` – Create one thread for each vehicle.
- `pthread_join()` – Wait for all vehicle threads.
- `pthread_mutex_t` – Protect shared bridge variables.
- `pthread_cond_t` – Make vehicles wait when the bridge cannot be entered.
- Shared variables:
 - `vehicles_on_bridge`
 - `current_direction`
- `pthread_cond_wait()` – Wait if:
 - Bridge already has 3 vehicles.
 - Vehicles from the opposite direction are crossing.

- `pthread_cond_broadcast()` – Notify waiting vehicles when the bridge becomes available.

2. Airport Runway Allocation

Problem

An airport has **2 runways**. Multiple aircraft request permission to land.

Emergency aircraft should be given priority over normal aircraft.

Sample Input

Number of aircraft: 5

Number of runways: 2

Aircraft	Type
A1	Normal
A2	Emergency
A3	Normal
A4	Emergency
A5	Normal

Sample Output

A2 (Emergency) assigned Runway 1

A4 (Emergency) assigned Runway 2

A2 completed landing and released Runway 1

A1 assigned Runway 1

A4 completed landing and released Runway 2

A3 assigned Runway 2

All aircraft landed successfully.

Students must use

- `pthread_create()` – One thread per aircraft.
- `pthread_mutex_t` – Protect runway availability.
- `pthread_cond_t` – Separate condition variables for:
 - Emergency aircraft
 - Normal aircraft
- Shared variables:
 - `available_runways`
 - `waiting_emergency`
 - `waiting_normal`
- `pthread_cond_wait()` – Aircraft waits when no runway is available.
- `pthread_cond_signal()` or `pthread_cond_broadcast()` – Notify waiting aircraft.
- Priority logic:

- Normal aircraft should wait if an emergency aircraft is waiting.

3. Parking Lot Management

Problem

A parking lot has **3 parking spaces** and **5 cars**. A car must wait if the parking lot is full.

Sample Input

Parking slots: 3
Number of cars: 5

Sample Output

Car 1 entered. Available slots: 2
Car 2 entered. Available slots: 1
Car 3 entered. Available slots: 0

Car 4 is waiting
Car 5 is waiting

Car 2 left the parking lot
Car 4 entered. Available slots: 0

Car 1 left the parking lot
Car 5 entered. Available slots: 0

Students must use

- pthread_create() – One thread for each car.
- pthread_mutex_t – Protect the parking slot count.
- One pthread_cond_t – For waiting cars.
- Shared variable:
 - available_slots
- pthread_cond_wait() – When available_slots == 0.
- pthread_cond_signal() – When a car leaves and releases a slot.
- pthread_join() – Wait for all cars.

4. Elevator Capacity Control

Problem

An elevator can accommodate a maximum of **4 passengers**. There are **7 passenger threads**.

A passenger must wait when the elevator is full.

Sample Input

Elevator capacity: 4
Number of passengers: 7

Sample Output

P1 entered the elevator
P2 entered the elevator
P3 entered the elevator
P4 entered the elevator

Elevator is full

P5 waiting
P6 waiting
P7 waiting

P1 exited
P5 entered

P2 exited
P6 entered

P3 exited
P7 entered

Students must use

- pthread_create() – One thread per passenger.
- pthread_mutex_t – Protect elevator occupancy.
- pthread_cond_t – Manage waiting passengers.
- Shared variable:
 - current_passengers
- pthread_cond_wait() – When elevator is full.
- pthread_cond_signal() – When a passenger exits.

Additional condition

current_passengers < MAX_CAPACITY

5. Railway Junction Synchronization

Problem

Trains arrive from **Track A and Track B**. Only **one train** can use the railway junction at a time.

The solution should ensure that trains from one track do not permanently block trains from the other track.

Sample Input

Number of trains: 5

Train	Track
T1	A
T2	B
T3	A
T4	B
T5	A

Sample Output

T1 from Track A entered the junction

T2 waiting from Track B
T3 waiting from Track A

T1 crossed the junction

T2 from Track B entered the junction
T2 crossed the junction

T3 from Track A entered the junction
T3 crossed the junction

All trains crossed safely.

Students must use

- `pthread_create()` – One thread for each train.
- `pthread_mutex_t` – Protect the railway junction.
- Two condition variables:
 - `trackA_cond`
 - `trackB_cond`
- Shared variables:
 - `junction_busy`
 - `last_track`
 - `waiting_A`
 - `waiting_B`
- `pthread_cond_wait()` – When the junction is occupied.
- `pthread_cond_signal()` – Notify the next waiting train.
- Alternation/fairness logic should be implemented to reduce starvation.

6. Operating Room Allocation with Priority

Problem

A hospital has **2 operating rooms**. Patients can be either **Emergency** or **Normal**.

Emergency patients should receive priority when an operating room becomes available.

Sample Input

Operating rooms: 2

Number of patients: 5

Patient	Type
---------	------

P1	Normal
----	--------

P2	Emergency
----	-----------

P3	Normal
----	--------

P4	Emergency
----	-----------

P5	Normal
----	--------

Sample Output

P2 (Emergency) assigned Operating Room 1

P4 (Emergency) assigned Operating Room 2

P2 completed surgery

P1 assigned Operating Room 1

P4 completed surgery

P3 assigned Operating Room 2

Students must use

- `pthread_create()` – One thread per patient.
- `pthread_mutex_t` – Protect operating room allocation.
- Two condition variables:
 - `emergency_cond`
 - `normal_cond`
- Shared variables:
 - `available_rooms`
 - `waiting_emergency`
 - `waiting_normal`
- Emergency patients should be signaled first.
- `pthread_cond_wait()` – Wait if no room is available.

7. Database Connection Pool

Problem

A system has **3 database connections** shared among **6 client threads**.

Each client must acquire one available connection, perform its operation, and release the connection.

Sample Input

Number of database connections: 3

Number of clients: 6

Sample Output

Client 1 acquired Connection 1

Client 2 acquired Connection 2

Client 3 acquired Connection 3

Client 4 waiting for connection

Client 5 waiting for connection

Client 6 waiting for connection

Client 2 released Connection 2

Client 4 acquired Connection 2

Client 1 released Connection 1

Client 5 acquired Connection 1

Students must use

- `pthread_create()` – One thread for each client.
- `pthread_mutex_t` – Protect the connection pool.
- `pthread_cond_t` – For clients waiting for a connection.
- Shared data structure:

```
int connection_status[3];
```

For example:

0 → Available

1 → Busy

- `pthread_cond_wait()` – When no connection is available.
- `pthread_cond_signal()` – When a connection is released.

Important

Students must ensure that **the same connection is not assigned to two clients simultaneously**.

8. Computer Lab Allocation with Priority

Problem

A computer lab has **3 computers** shared by **students and faculty**.

Faculty members should receive priority over waiting students.

Sample Input

Number of computers: 3

Number of users: 6

User	Type
U1	Student
U2	Faculty
U3	Student
U4	Faculty
U5	Student
U6	Student

Sample Output

U2 (Faculty) assigned Computer 1
U4 (Faculty) assigned Computer 2
U1 (Student) assigned Computer 3

U3 waiting
U5 waiting
U6 waiting

U2 released Computer 1
U3 assigned Computer 1

U4 released Computer 2
U5 assigned Computer 2

Students must use

- pthread_create() – One thread per user.
- pthread_mutex_t – Protect computer allocation.
- Two condition variables:
 - faculty_cond
 - student_cond
- Shared variables:

- available_computers
 - waiting_faculty
 - waiting_students
- Priority condition:
 - A student should wait if a faculty member is waiting.

9. Traffic Intersection Synchronization

Problem

Vehicles arrive from **North, South, East, and West**.

Rules:

- North and South vehicles may pass simultaneously.
- East and West vehicles may pass simultaneously.
- North/South traffic cannot pass together with East/West traffic.

Sample Input

Number of vehicles: 6

Vehicle	Direction
V1	North
V2	South
V3	East
V4	West
V5	North
V6	East

Sample Output

V1 entered from North
V2 entered from South

V3 waiting from East
V4 waiting from West

V1 exited
V2 exited

V3 entered from East
V4 entered from West

V3 exited
V4 exited

V5 entered from North

Students must use

- pthread_create() – One thread per vehicle.
- pthread_mutex_t – Protect intersection state.
- At least **two condition variables**:
 - north_south_cond
 - east_west_cond
- Shared variables:
 - ns_count
 - ew_count
- pthread_cond_wait() – When conflicting traffic is using the intersection.
- pthread_cond_broadcast() – When an entire direction group finishes.

This is one of the **more challenging questions** because students must identify **compatible and conflicting threads**.

10. Shared Resource Pool

Problem

A system contains **3 identical resources** shared among **7 threads**.

Each thread must:

1. Request a resource.
2. Use the resource.
3. Release the resource.

Sample Input

Number of resources: 3

Number of threads: 7

Sample Output

T1 acquired Resource 1

T2 acquired Resource 2

T3 acquired Resource 3

T4 waiting

T5 waiting

T6 waiting

T7 waiting

T2 released Resource 2

T4 acquired Resource 2

T1 released Resource 1

T5 acquired Resource 1

T3 released Resource 3

T6 acquired Resource 3

Students must use

- `pthread_create()` – Create 7 worker threads.
- `pthread_join()` – Wait for completion.
- `pthread_mutex_t` – Protect the resource pool.
- `pthread_cond_t` – Wait for resource availability.
- Shared array:

```
int resource[3];
```

Example:

`resource[i] = 0` → Free

`resource[i] = 1` → Allocated

- `pthread_cond_wait()` – If all resources are busy.
- `pthread_cond_signal()` – When a resource is released.